

Block-Jacobi and band-solve preconditioners for the cutDG multiphase mass system

MeltPoolDG – compressible multiphase cutDG solver

October 2026

Abstract

In every explicit Euler step, the compressible multiphase cutDG solver solves a linear system with a cut DG mass matrix stabilized by ghost-penalty terms. These notes summarize two matrix-free preconditioners for this system: a block-Jacobi preconditioner (**BlockJacobi**) that inverts the cell-wise and phase-wise diagonal blocks, and a band-solve preconditioner (**BandSolve**) that exploits an exact decoupling of the system into cells away from the interface and a narrow band of cells around it. Neither preconditioner assembles a matrix, and their setup cost scales with the number of cells near the interface rather than with the total number of cells.

Contents

1	Problem setting	1
2	Block-Jacobi preconditioner (BlockJacobi)	3
2.1	Blocks	3
2.2	Band cells and outer cells	3
2.3	Spectral properties	4
2.4	Cost	4
3	Exact decoupling of the system	4
4	Band-solve preconditioner (BandSolve)	5
4.1	Definition	5
4.2	Inner tolerance and outer convergence	5
4.3	Implementation	6
4.4	Cost and scalability	6
4.5	Alternatives considered	7
5	Interaction with the DoF layout and the update frequency	7
6	Measured results	7
7	Usage and implementation overview	8

1 Problem setting

Mesh and phases. Let $\mathcal{T}_h$ be the mesh and ϕ_h the discrete level-set function. With respect to the zero level set Γ , each cell $K \in \mathcal{T}_h$ is classified as *liquid* (inside), *intersected* or *gas* (outside);

this classification defines the active FE index of the cell (0, 1, 2 in `CutUtil::CellCategory`). We write $\mathcal{T}_\Gamma$ for the intersected cells and, for a phase $q \in \{\ell, g\}$,

$$\mathcal{T}_h^q = \{K \in \mathcal{T}_h : K \text{ is a bulk cell of phase } q\} \cup \mathcal{T}_\Gamma$$

for the active mesh of phase q . Each phase carries $d + 2$ conserved variables, discretized with FE_DGQ elements of degree p on the cells of $\mathcal{T}_h^q$ and FE_Nothing elsewhere. Every active cell therefore holds

$$n_s = (p + 1)^d$$

scalar DoFs per conserved variable and phase.

Linear system. In each explicit Euler step, the new state U^{n+1} solves

$$a(U^{n+1}, V) = \ell^n(V) \quad \forall V, \quad (1)$$

where ℓ^n collects the old mass term and the explicitly evaluated fluxes, and

$$a(W, V) = \sum_{q \in \{\ell, g\}} \left[\sum_{K \in \mathcal{T}_h^q} \int_{K \cap \Omega_q} W_q \cdot V_q \, dx + \sum_{F \in \mathcal{F}_q^{\text{GP}}} g_F(W_q, V_q) \right]. \quad (2)$$

On bulk cells, the first term is the standard DG mass matrix; on intersected cells, it is the cut mass matrix integrated with the non-matching quadrature of `NonMatching::MappingInfo`.

Ghost penalty. The ghost-penalty faces of phase q are the interior faces with phase q present on both sides and at least one intersected neighbor,

$$\mathcal{F}_q^{\text{GP}} = \{F = \partial K^- \cap \partial K^+ : K^\pm \in \mathcal{T}_h^q, K^- \in \mathcal{T}_\Gamma \text{ or } K^+ \in \mathcal{T}_\Gamma\}, \quad (3)$$

as implemented in `CutUtil::face_type_has_ghost_penalty()`. On these faces,

$$g_F(w, v) = \sum_{j=0}^J \gamma_j h^{2j+1} \int_F \llbracket \partial_n^j w \rrbracket \llbracket \partial_n^j v \rrbracket \, ds, \quad J = \begin{cases} 2, & p = 2, \\ 1, & \text{otherwise,} \end{cases} \quad (4)$$

with $\llbracket \cdot \rrbracket = (\cdot)^- - (\cdot)^+$, $\partial_n^0 w = w$, $\partial_n^1 w = \nabla w \cdot n$, $\partial_n^2 w = n^\top \nabla^2 w n$, penalty parameters γ_j (`gamma_M_degree_j`) and $h = h_{\min}$ the global minimum cell size, as in `ghost_penalty_face_integral()`.

Algebraic structure. The form (2) acts identically on all conserved variables and never couples the two phases. With $A^q \in \mathbb{R}^{N_q \times N_q}$ the scalar matrix of phase q , the system matrix is

$$A = (A^\ell \otimes I_{d+2}) \oplus (A^g \otimes I_{d+2}). \quad (5)$$

A is symmetric positive definite and well conditioned: the condition number of a DG mass matrix depends on p but not on h (quasi-uniform meshes), and the ghost penalty keeps it bounded independently of how the interface cuts the cells. Unpreconditioned CG therefore already converges in a moderate, mesh-independent number of iterations (31 in the test of Section 6).

Consequences for preconditioning.

- Matrix-based preconditioners (AMG, ILU) must assemble the matrix from the matrix-free operator and set up the hierarchy or factorization. Whenever a cell changes its category, the DoF layout changes and this setup has to be repeated, which for a moving interface happens almost every time step. Moreover, AMG targets smooth low-energy error modes of elliptic operators, which a mass matrix does not have.
- A preconditioner can save at most the ≈ 30 operator applications of unpreconditioned CG. It must be cheap to set up and to apply, and its setup should not scale with the whole mesh.

2 Block-Jacobi preconditioner (BlockJacobi)

2.1 Blocks

A block contains the n_s DoFs of one scalar component of one phase on one cell. Because of (5), one scalar block per cell and phase suffices; it is applied to all $d + 2$ conserved variables. For $K \in \mathcal{T}_h^q$ with Lagrange basis $\{\varphi_i\}_{i=1}^{n_s}$ (lexicographic numbering of FE_DGQ, identical to the numbering inside `FEEvaluation`), the diagonal block is

$$(A_{KK}^q)_{ij} = \underbrace{\int_{K \cap \Omega_q} \varphi_j \varphi_i \, dx}_{(M_K^q)_{ij}} + \sum_{\substack{F \in \mathcal{F}_q^{\text{GP}} \\ F \subset \partial K}} \underbrace{\int_F \sum_{l=0}^J \gamma_l h^{2l+1} \partial_n^l \varphi_j \partial_n^l \varphi_i \, ds}_{(G_F^{KK})_{ij}}. \quad (6)$$

The face term is the self-coupling part of (4): if test and trial function both live on K , each jump reduces to $\pm \partial_n^l \varphi$ and the product of the two jumps is positive. Since $\partial_n^1 \varphi_i \partial_n^1 \varphi_j$ and $\partial_n^2 \varphi_i \partial_n^2 \varphi_j$ are invariant under $n \mapsto -n$, G_F^{KK} depends neither on the neighbor nor on the orientation of the normal vector and can be computed cell-locally.

The block-Jacobi preconditioner is

$$P_{\text{BJ}} = \bigoplus_q \bigoplus_{K \in \mathcal{T}_h^q} A_{KK}^q \otimes I_{d+2}. \quad (7)$$

2.2 Band cells and outer cells

We split the active cells of each phase into the *band* and the *outer cells*,

$$\mathcal{B}_q = \mathcal{T}_\Gamma \cup \{K \in \mathcal{T}_h^q \setminus \mathcal{T}_\Gamma : \exists F \in \mathcal{F}_q^{\text{GP}} \text{ with } F \subset \partial K\}, \quad (8)$$

$$\mathcal{O}_q = \mathcal{T}_h^q \setminus \mathcal{B}_q. \quad (9)$$

For $K \in \mathcal{O}_q$ the block (6) is just the cell mass matrix, $A_{KK}^q = M_K$. Only band cells need a dense block.

Outer cells: inverse mass matrix by sum factorization. With the Gauss quadrature of $n_q = p + 1$ points per direction used by the operator,

$$M_K = S^\top W_K S, \quad S_{\mu i} = \varphi_i(\hat{x}_\mu), \quad W_K = \text{diag}(|\det J_K(\hat{x}_\mu)| w_\mu), \quad (10)$$

where $S = S_1 \otimes \cdots \otimes S_1$ is square and invertible. Hence

$$M_K^{-1} = S^{-1} W_K^{-1} S^{-\top}, \quad (11)$$

which is applied by sum factorization with the inverse 1D shape matrices (`MatrixFreeOperators::CellwiseInverseMassMatrix`). This is the exact inverse of the quadrature-based mass matrix of the operator, also on curved cells. It costs about one mass operator application and needs neither setup nor storage. The implementation checks that $n_q = p + 1$.

Band cells: dense blocks. For $K \in \mathcal{B}_q$, the block (6) is assembled column by column:

- mass part of bulk band cells: the cell mass operator applied to unit vectors e_j (`FEEvaluation`, vectorized over the cells of a batch);
- cut mass part of intersected cells: evaluation and integration of e_j with the non-matching quadrature of phase q (`FEPPointEvaluation` on `mapping_info_cells[q]`);

- ghost-penalty part: **FEFaceValues** on the faces of K , or **FESubfaceValues** if the neighbor is refined, with the same face quadrature and mapping as the matrix-free face loop. Periodic neighbors are supported.

Each block is inverted by Gauss–Jordan elimination with partial pivoting. A numerically singular block, which is only expected for a singular system (e.g. a vanishing cut without stabilization), is replaced by its inverse diagonal and a warning is printed.

2.3 Spectral properties

Since outer cells do not couple to any other cell (Lemma 1), the preconditioned operator decomposes as

$$P_{\text{BJ}}^{-1}A = I_{\mathcal{O}} \oplus P_{\mathcal{B}}^{-1}A_{\mathcal{B}}, \quad (12)$$

where $A_{\mathcal{B}}$ is the band matrix and $P_{\mathcal{B}}$ its block diagonal. All eigenvalues associated with the outer cells are exactly one; the remaining spectrum stems from the ghost-penalty coupling between neighboring band cells. CG convergence is governed by this small part of the spectrum, which explains why the iteration count drops to that of AMG (11) although the estimated condition number (≈ 10) is larger than for AMG (≈ 2.3).

2.4 Cost

- *Setup* (**update()**): one pass over the faces of all cells to identify the band, plus $\mathcal{O}(n_s)$ cell and face evaluations and an $\mathcal{O}(n_s^3)$ inversion per band cell and phase.
- *Memory*: n_s^2 values per band cell and phase (64 for $p = 1$, 729 for $p = 2$ in 3D).
- *Application*: one inverse mass application on all bulk cells ($\approx$ one operator application) plus a dense $n_s \times n_s$ product per band cell, phase and conserved variable.

One preconditioned CG iteration therefore costs about two operator applications: with 11 instead of 31 iterations, the solve costs about 22 instead of 31 operator applications, before counting the setup.

3 Exact decoupling of the system

Lemma 1 (Decoupling). *For each phase q , every ghost-penalty face lies inside the band: if $F = \partial K^- \cap \partial K^+ \in \mathcal{F}_q^{\text{GP}}$, then $K^-, K^+ \in \mathcal{B}_q$. Consequently, after a permutation of the DoFs,*

$$A^q = \left(\bigoplus_{K \in \mathcal{O}_q} M_K \right) \oplus A_{\mathcal{B}}^q, \quad (13)$$

where $A_{\mathcal{B}}^q$ contains the (cut) mass matrices of the band cells and all ghost-penalty terms.

Proof. If $K^\pm$ is intersected, it belongs to $\mathcal{B}_q$ by (8). Otherwise it is a bulk cell of phase q with the ghost-penalty face $F \subset \partial K^\pm$, so it belongs to $\mathcal{B}_q$ as well. The DG mass matrix couples only DoFs of the same cell, and the only face terms of (2) are the ghost-penalty terms. Hence an outer cell couples to no other cell, and band cells couple only among themselves. $\square$

Consequently,

$$A^{-1}r = (M_{\mathcal{O}}^{-1}r_{\mathcal{O}}) \oplus (A_{\mathcal{B}}^{-1}r_{\mathcal{B}}), \quad (14)$$

where $M_{\mathcal{O}}^{-1}$ is the cell-wise inverse mass matrix of Section 2. The outer part is solved exactly by one sum-factorized application. Only the band system $A_{\mathcal{B}}$ requires an iterative (or direct) solver. Its size is proportional to the number of cells near the interface: for a band of fixed width in cells, $|\mathcal{B}|/|\mathcal{T}_h| = \mathcal{O}(h) = \mathcal{O}(N^{-1/d})$.

Remark 1. The decoupling also explains (12): block-Jacobi is exact on $\mathcal{O}$, and all remaining CG iterations are spent on the band, while every iteration still processes all cells. *BandSolve* removes this mismatch.

4 Band-solve preconditioner (BandSolve)

4.1 Definition

Based on (14), the band-solve preconditioner is

$$P_{\text{BS}}^{-1}r = (M_{\mathcal{O}}^{-1}r_{\mathcal{O}}) \oplus \text{PCG}_{\tau}(A_{\mathcal{B}}, P_{\mathcal{B}}; r_{\mathcal{B}}), \quad (15)$$

where $\text{PCG}_{\tau}(A_{\mathcal{B}}, P_{\mathcal{B}}; r_{\mathcal{B}})$ denotes the approximate solution of $A_{\mathcal{B}}x = r_{\mathcal{B}}$ by the conjugate gradient method, preconditioned with the dense block inverses of the band and stopped at tolerance τ . With an exact inner solve, $P_{\text{BS}} = A$, and the outer CG converges in one iteration. Since the inner CG works on the band only, both the operator applications and the vector operations of the inner solver scale with $|\mathcal{B}|$, not with $|\mathcal{T}_h|$.

Algorithm 1 Application $z = P_{\text{BS}}^{-1}r$ (`BandSolvePreconditioner::vmult()`)

```

1:  $z \leftarrow P_{\text{BJ}}^{-1}r$  ▷ exact on  $\mathcal{O}$ ; band entries are overwritten below
2:  $x \leftarrow 0, \quad s \leftarrow r_{\mathcal{B}}$  ▷ band vectors
3:  $\tau \leftarrow \max(\frac{1}{2}\varepsilon_{\text{rel}}\|s\|, \frac{1}{2}\varepsilon_{\text{abs}}, 10^{-14}\|s\|)$ 
4: if  $\|s\| \leq \tau$  then
5:    $z_{\mathcal{B}} \leftarrow 0$ ; return
6: end if
7:  $y \leftarrow P_{\mathcal{B}}^{-1}s, \quad d \leftarrow y, \quad \rho \leftarrow s^{\top}y$ 
8: for  $k = 1, \dots, k_{\text{max}}$  do ▷  $k_{\text{max}} = \min(200, \text{outer max. iterations})$ 
9:    $w \leftarrow A_{\mathcal{B}}d$  ▷ vmult_band(): band cells and ghost-penalty faces
10:   $\sigma \leftarrow d^{\top}w$ ; if  $\sigma \leq 0$  break ▷ safeguard
11:   $\alpha \leftarrow \rho/\sigma, \quad x \leftarrow x + \alpha d, \quad s \leftarrow s - \alpha w$ 
12:   $y \leftarrow P_{\mathcal{B}}^{-1}s$  ▷ apply_inverse_block_diagonal_band()
13:   $(\rho_{\text{new}}, \|s\|^2) \leftarrow (s^{\top}y, s^{\top}s)$  ▷ one reduction
14:  if  $\|s\| \leq \tau$  then break
15:  end if
16:   $d \leftarrow y + (\rho_{\text{new}}/\rho)d, \quad \rho \leftarrow \rho_{\text{new}}$ 
17: end for
18:  $z_{\mathcal{B}} \leftarrow x$ 

```

4.2 Inner tolerance and outer convergence

The inner tolerance is derived from the tolerances $\varepsilon_{\text{rel}}, \varepsilon_{\text{abs}}$ of the outer solver (Algorithm 1, line 3). The factor $\frac{1}{2}$ makes the inner residual smaller than the outer target. The floor $10^{-14}\|s\|$ avoids iterating below the attainable accuracy in double precision.

Why one outer iteration suffices. Let the inner solver leave the band residual $e = r_{\mathcal{B}} - A_{\mathcal{B}}z_{\mathcal{B}}$, and let $\hat{e}$ be e extended by zero. Then $z = A^{-1}(r_0 - \hat{e})$, and the first outer CG step gives

$$\alpha = \frac{r_0^{\top}z}{z^{\top}Az} = 1 + \frac{\hat{e}^{\top}A^{-1}(r_0 - \hat{e})}{(r_0 - \hat{e})^{\top}A^{-1}(r_0 - \hat{e})}, \quad r_1 = r_0 - \alpha Az = (1 - \alpha)r_0 + \alpha \hat{e}. \quad (16)$$

By the Cauchy–Schwarz inequality, $|1 - \alpha| \leq \|\hat{e}\|_{A^{-1}}/\|r_0 - \hat{e}\|_{A^{-1}}$, so

$$\|r_1\| \lesssim (1 + C\sqrt{\kappa(A)})\|e\|, \quad C = \|r_0\|/\|r_0 - \hat{e}\| \approx 1. \quad (17)$$

The outer residual after one iteration is thus of the size of the inner residual, up to a factor that depends on $\sqrt{\kappa(A)}$. In practice, one outer iteration is expected, occasionally two.

Remark 2 (Variable preconditioning). *Because the inner solve stops at a tolerance, P_{BS}^{-1} depends slightly nonlinearly on its argument. With one or two outer iterations this has no practical effect. If many outer iterations were observed, a flexible CG variant would be the safe choice.*

4.3 Implementation

- *Band definition:* a cell (and phase) belongs to the band if and only if `compute_inverse_block_diagonal_from_matrixfree()` stored a dense block for it. `CellwiseBlockInverses` additionally records which cell batches contain band cells.
- *Band DoFs* (`extract_band_dof_indices()`): a marker vector is set to one on the band cells via `FEEvaluation::set_dof_values_plain()` with a lane mask; the local indices of the marked entries form the index list used by all inner vector operations.
- *Band operator* (`vmult_band()`): a `MatrixFree::loop()` without zeroing the destination. The cell worker processes only cell batches with band cells (contiguous runs are passed to `local_apply_cell_lhs()`), and the face worker is the regular ghost-penalty face worker. Bulk lanes outside the band contribute zero because the input vector vanishes outside the band.
- *Invariant:* the four work vectors of the inner solver are full-size `LinearAlgebra::distributed::Vectors` for compatibility with `MatrixFree`, but they are zero outside the band, and only band entries are ever written by the inner solver.
- *Monitoring:* the inner iteration counts are recorded in the `IterationMonitor` under the label `band_solve`.

4.4 Cost and scalability

Let C_A denote the cost of one full operator application, $f_{\mathcal{B}} = |\mathcal{B}|/|\mathcal{T}_h|$ the band fraction, n_{out} the number of outer and n_{in} the number of inner iterations. Per linear solve,

$$C_{\text{BS}} \approx (1 + n_{\text{out}}) C_A + n_{\text{out}} C_{M^{-1}} + n_{\text{out}} n_{\text{in}} (C_{A_{\mathcal{B}}} + C_{P_{\mathcal{B}}}), \quad C_{A_{\mathcal{B}}}, C_{P_{\mathcal{B}}} = \mathcal{O}(f_{\mathcal{B}} C_A), \quad (18)$$

where the first term accounts for the initial residual of the outer CG. With $C_{M^{-1}} \approx C_A$, $n_{\text{out}} \approx 1$ and $n_{\text{in}} \approx 11$, this is about $3 + 22f_{\mathcal{B}}$ operator applications. Table 1 compares the estimates.

Table 1: Estimated cost of one linear solve in units of full operator applications (vector operations and setup not included).

Preconditioner	Iterations	Cost estimate
Identity	31	≈ 31
Diagonal	19	$\approx 19 + \text{diagonal setup}$
BlockJacobi	11	$\approx 22 + \text{band setup}$
AMG / ILU	11 / 15	iterations + matrix assembly and setup (dominant)
BandSolve	1–2 outer, ≈ 11 inner	$\approx 3 + 22f_{\mathcal{B}} + \text{band setup}$

Scalability.

- The inner iteration count is expected to be independent of h , since the ghost penalty bounds the conditioning of the band system independently of mesh size and cut position.
- The band fraction decreases under refinement, $f_B = \mathcal{O}(N^{-1/d})$, so the advantage over **BlockJacobi** grows with the problem size.
- Communication per inner iteration: two global reductions and one ghost exchange. The ghost exchange currently covers the full ghost layer; restricting it to band DoFs (with a separate partitioner) is the next optimization if profiling on many ranks shows a need.
- Ranks without band cells wait during the inner solve. Weighting intersected cells in the load balancing keeps this imbalance small.

4.5 Alternatives considered

- *Additive Schwarz* with patches of a cut cell and its ghost-penalty neighbors would capture the neighbor coupling and reduce the iteration count to perhaps 5–7. However, it has a larger setup $((1+2d)n_s$ unknowns per patch), overlapping and more expensive applications, patches crossing MPI boundaries, and it needs the symmetric variant for CG. Every iteration still processes all cells.
- *Direct solve of the band* (e.g. Amesos/KLU, MUMPS) would make P_{BS} exact. However, the band grows like $N^{(d-1)/d}$, it has to be refactorized almost every step, KLU gathers the matrix on one rank, and distributed direct solvers scale poorly to many ranks. The iterative band solve was therefore chosen for large-scale runs.

5 Interaction with the DoF layout and the update frequency

- The DoF layout is fully determined by the cell categories. If no cell changes its category (`CutUtil::cell_locations_changed()`), the solution transfer, `setup_dof_system()` and the preconditioner `reinit()` are skipped. Only the cut quadrature is recomputed.
- `reinit()` clears the block data, because the blocks are addressed by the matrix-free cell-batch numbering, which changes with the DoF layout. The subsequent `update()` is enforced by `preconditioner_update_flag`.
- Between layout changes, only the values change (cut geometry). Outdated blocks remain symmetric positive definite, so `preconditioner update frequency > 1` is admissible for both preconditioners at the price of slightly more iterations.

6 Measured results

Table 2 lists the results of the first test case (524 288 DoFs, solver tolerance 10^{-18} , preconditioner update frequency 50). The condition numbers are CG-based estimates. The run times include more than the linear solve; their near independence of the iteration count (Identity vs. BlockJacobi) indicates that the linear solve is not the dominant cost of a time step in this test.

Recommended checks for **BandSolve**.

1. The solution agrees with **BlockJacobi** up to the solver tolerance.
2. The outer CG (`linear_solve`) takes 1–2 iterations, and the inner solver (`band_solve`) takes about as many iterations as **BlockJacobi**.

Table 2: Iterations, run time and estimated condition number for the test case.

Preconditioner	Iterations	Time [s]	Condition number
Identity	31	4.18	75.02
Diagonal	19	3.89	20.49
AMG	11	21.53	2.32
ILU	15	10.74	12.37
BlockJacobi	11	4.16	10.15
BandSolve	to be measured		

3. A timing breakdown (`preconditioner.update()`, linear solve, `create_rhs()`, `adapt_to_new_interface_position()`) shows the share of the linear solve.

7 Usage and implementation overview

Parameters.

```
"linear solver": {
  "preconditioner type": "BandSolve",
  "solver type": "CG"
}
```

Use "BlockJacobi" for the block-Jacobi preconditioner. Requirements: FE_DGQ, cell quadrature with $p + 1$ points per direction, matrix-free operator. For BandSolve, the inner tolerances are derived from the outer `rel tolerance`, `abs tolerance` and `max iterations`.

Files and interfaces.

- `linear_algebra/preconditioner_block_jacobi.hpp`: storage `CellwiseBlockInverses`, concept `BlockJacobiPreconditionerOperatorType`, class `BlockJacobiPreconditioner`.
- `linear_algebra/preconditioner_band_solve.hpp`: concept `BandSolvePreconditionerOperatorType`, class `BandSolvePreconditioner` (inner PCG).
- `linear_algebra/preconditioner_factory.hpp`, `linear_solver_data.hpp`: enum values `BlockJacobi`, `BandSolve`; optional argument `linear_solver_data` of `make_preconditioner()`.
- `compressible_flow/multiphase_operator.hpp/.cpp`: operator interface `compute_inverse_block_diagonal_from_matrixfree()`, `apply_inverse_block_diagonal()`, `apply_inverse_block_diagonal_band()`, `vmult_band()`, `extract_band_dof_indices()`.

Open points.

- Ghost exchange restricted to band DoFs (separate partitioner).
- Grouping band cells into separate cell batches (additional FE indices) to avoid partially filled SIMD lanes in the band operator.
- Reuse of the dense blocks of bulk band cells between updates; they change only with the DoF layout.
- Local instead of global cell size h in the ghost-penalty scaling (marked as TODO in the operator; the preconditioner uses the same h as the operator).